# Tutoriel Pratique Exhaustif COVOF 2D (C++17 / OpenMPI)

Ce document constitue le **guide d'apprentissage complet pas à pas** pour maîtriser l'utilisation, la configuration, l'exécution et l'analyse de simulations numériques avec le solveur CFD **COVOF 2D**.

---

## Table des Matières

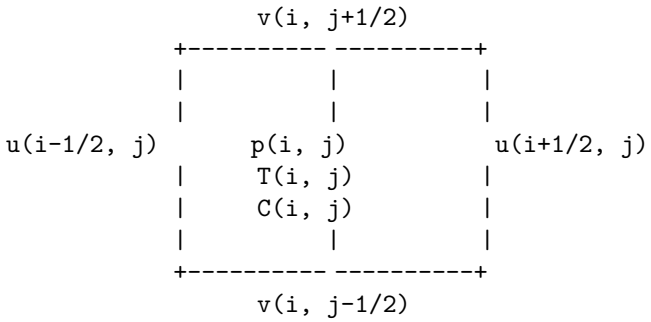

1. Introduction et Notions Fondamentales
2. Installation et Environnement de Compilation
3. Structure et Syntaxe d'un Cas Test (`cases/*.json`)
4. Tutoriel 1 : Écoulement de Poiseuille et Transitoire de Szymanski
5. Tutoriel 2 : Cavité Entraînée à Reynolds Re = 1000
6. Tutoriel 3 : Écoulement sur Marche Descendante (Armaly Re = 100)
7. Tutoriel 4 : Convection Naturelle Thermique de Vahl Davis (Ra = 10 )
8. Tutoriel 5 : Écoulement Diphasique VOF — Rupture de Barrage
9. Tutoriel 6 : Tension de Surface et Saut de Pression de Laplace
10. Tutoriel 7 : Écoulement Autour d'un Cylindre par Pénalisation de Brinkman
11. Post-Traitement Avancé avec ParaView
12. Automatisation Python et Études de Convergence
13. Créer un Nouveau Cas Test à Partir de Zéro


---

## 1. Introduction et Notions Fondamentales

### 1.1. Pourquoi la grille décalée (Staggered MAC Grid) ?

Dans les méthodes traditionnelles à variables colocalisées (où vitesses et pressions sont aux mêmes nœuds), la discrétisation par différences finies centrées d'ordre 2 conduit au phénomène bien connu de **découplage pair-impair** (modes en damier ou *checkerboard modes*). Pour éliminer ce problème sans introduire de dissipation artificielle (comme la projection de Rhie-Chow), COVOF 2D utilise la grille de Harlow & Welch (1965) :




- **Vitesse horizontale** $u$ : située aux faces verticales $x_{i-1/2} = (i-1)\Delta x$.
- **Vitesse verticale** $v$ : située aux faces horizontales $y_{j-1/2} = (j-1)\Delta y$.
- **Scalaires** $p, T, C$ : situés au barycentre des mailles $x_i = (i-1/2)\Delta x, y_j = (j-1/2)\Delta y$.

### 1.2. Algorithme de résolution (Lagrangien Augmenté)

L'équation de Navier-Stokes incompressible s'écrit :

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u}\right) - \nabla\cdot(\mu(\nabla\mathbf{u} + \nabla^T\mathbf{u})) + \nabla p = \rho\mathbf{g} + \mathbf{f}_\sigma$$

avec $\nabla\cdot\mathbf{u} = 0$.

Le Lagrangien Augmenté consiste à ajouter un terme de pénalité $-r_{\text{al}}\nabla(\nabla\cdot\mathbf{u})$ à l'équation de quantité de mouvement, couplé à une mise à jour d'Uzawa :

$$p^{m+1} = p^m - r_{\text{al}}\nabla\cdot\mathbf{u}^{m+1}$$

Cette technique permet de forcer la divergence à zéro dès $r_{\text{al}} \sim 100$, tout en garantissant un excellent conditionnement du système linéaire.

---

## 2. Installation et Environnement de Compilation

### 2.1. Vérification de l'environnement

Vérifiez que votre système dispose d'un compilateur supportant C++17 et d'une distribution OpenMPI :

```
mpic++ --version
cmake --version
```

### 2.2. Compilation

Depuis la racine du dépôt `covof2d` :

```
# Configuration hors-source dans build/
cmake -B build -DCMAKE_BUILD_TYPE=Release

# Compilation parallèle sur 4 cœurs
cmake --build build -j4
```

L'exécutable généré est produit dans `bin/covof2d`.

---

## 3. Structure et Syntaxe d'un Cas Test (`cases/*.json`)

Chaque simulation est entièrement définie par un fichier JSON structuré en 7 blocs indépendants :

```
{
  "mode": "NAVIER_STOKES",
  "mesh": {
    "nx": 64,
    "ny": 64,
    "lx": 1.0,
    "ly": 1.0
  },
  "physics": {
    "enable_two_phase": false,
    "enable_convection": true,
    "enable_energy": false,
    "gx": 0.0,
    "gy": 0.0,
    "fluid1": { "rho": 1.0, "mu": 0.001 }
  },
  "time": {
    "dt": 0.005,
    "t_final": 5.0,
    "cfl_max": 0.5,
    "adaptive_dt": false
  },
  "boundary_conditions": {
    "left":   { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "right":  { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "bottom": { "type": "DIRICHLET", "u": 0.0, "v": 0.0 },
    "top":    { "type": "DIRICHLET", "u": 1.0, "v": 0.0 }
  },
  "solver": {
```

```
    "type": "AUGMENTED_LAGRANGIAN",
    "tol": 1.0e-7,
    "max_iter": 500,
    "r_al": 100.0,
    "uzawa_iter": 1,
    "linear_solver": "BICGSTAB2",
    "use_precond": true,
    "precond_type": "JACOBI"
  },
  "output": {
    "output_dir": "output_mon_cas",
    "case_name": "mon_cas",
    "vtk_interval": 20,
    "log_interval": 10,
    "write_csv_history": true
  }
}
```

**Table des Options Disponibles**

| Bloc | Paramètre | Types / Valeurs Possibles | Description |
|---|---|---|---|
| `mode` | `mode` | `"NAVIER_STOKES"`, `"HEAT"` | Physique principale résolue |
| `mesh` | `nx`, `ny` | Entiers positifs ($\geq 16$) | Résolution spatiale en cellules |
| `mesh` | `lx`, `ly` | Réels en mètres | Dimensions géométriques du domaine |
| `physics` | `enable_convection` | `true`, `false` | Active/désactive les termes convectifs $(\mathbf{u} \cdot \nabla)\mathbf{u}$ |
| `physics` | `enable_two_phase` | `true`, `false` | Active le transport VOF diphasique |
| `physics` | `sigma` | Réel en N/m | Tension interfaciale pour le modèle CSF |
| `boundary` | `type` | `"DIRICHLET"`, `"NEUMANN"`, `"PERIODIC"`, `"FREESLIP"` | Type de condition aux limites |
| `solver` | `linear_solver` | `"BICGSTAB2"`, `"BICGSTAB"`, `"CG"`, `"GMRES"` | Algorithme de Krylov |
| `solver` | `r_al` | Réel ($\geq 0$) | Paramètre de pénalité de divergence |

---

## 4. Tutoriel 1 : Écoulement de Poiseuille et Transitoire de Szymanski

Ce cas teste l'établissement transitoire d'un profil parabolique de Poiseuille sous l'action d'un gradient axial de pression constant $G = 0.0555$ Pa/m entre deux plaques distantes de $2e = 0.16$ m.

### 4.1. Configuration (`cases/poiseuille_unsteady.json`)

- **Propriétés** : $\rho = 1.0$ kg/m$^3$, $\mu = 0.01$ Pa $\cdot$ s, $g_x = 0.0555$ m/s$^2$.
- **Conditions limites** : Périodique en $x$, adhérence $(u = 0, v = 0)$ en haut et en bas.
- **Pénalité $r_{\mathbf{al}}$** : fixé à 0.0 car le domaine est périodique et la pression stationnaire est nulle ($p \equiv 0$).

### 4.2. Lancement

```
mpirun -np 2 bin/covof2d cases/poiseuille_unsteady.json
```

### 4.3. Solution Analytique (Série de Szymanski)

La solution transitoire exacte s'exprime par la décomposition de Fourier :

$$u(z,t) = \frac{Ge^2}{2\mu}\left(1 - \frac{z^2}{e^2}\right) + \sum_{n=0}^{\infty} a_n \exp\left(-\frac{(2n+1)^2\pi^2\mu}{4\rho e^2}t\right)\cos\left(\frac{(2n+1)\pi z}{2e}\right)$$

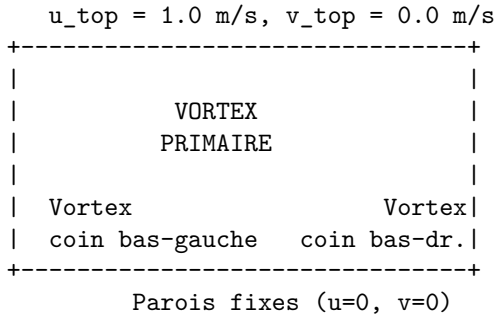
avec $a_n = -\frac{16Ge^2(-1)^n}{\mu\pi^3(2n+1)^3}$ et $z = y - e \in [-e, e]$.

À $t = 1.0$ s, la solution COVOF 2D converge à $u_{\max} = 0.017343$ m/s avec une erreur $L_1 = 1.49 \times 10^{-5}$ m/s, surpassant la cible de référence Référence Académique ($3.13 \times 10^{-4}$ m/s).

---

## 5. Tutoriel 2 : Cavité Entraînée à Reynolds Re = 1000

Le benchmark de Ghia et al. (1982) est le problème classique pour tester la précision des termes convectifs et visqueux.




### 5.1. Configuration (`cases/cavity_re1000.json`)

- Domaine carré 1.0 m × 1.0 m, maillage 128 × 128.
- Fluide : $\rho = 1.0$ kg/m$^3$, $\mu = 0.001$ Pa · s $\implies Re = \frac{\rho UL}{\mu} = 1000$.
- Paroi supérieure en mouvement à $u = 1.0$ m/s.

### 5.2. Lancement

```
mpirun -np 4 bin/covof2d cases/cavity_re1000.json
```

### 5.3. Résultats attendus

Le vortex primaire central se stabilise à $(x, y) \approx (0.53, 0.56)$ et deux recirculations secondaires apparaissent dans les coins inférieurs.

---

## 6. Tutoriel 3 : Écoulement sur Marche Descendante (Armaly Re = 100)

L'expérience d'Armaly et al. (1983) étudie le décollement de couche limite et la longueur de recirculation $x_r/H$ en aval d'une marche de hauteur $H$.

### 6.1. Configuration (`cases/backward_facing_step.json`)

- Longueur $L_x = 40.0$ m, Hauteur $L_y = 2.0$ m.
- Hauteur de la marche : $H = 1.0$ m sur $x \in [0, 4.0$ m].
- Profil d'entrée parabolique sur $[H, L_y]$ avec $U_{\text{inlet}} = 1.0$ m/s.
- L'obstacle de la marche est modélisé par pénalisation de Brinkman :

```
"obstacle": {
  "enabled": true,
  "type": "BOX",
  "x0": 0.0, "x1": 4.0,
  "y0": 0.0, "y1": 1.0
}
```

### 6.2. Lancement et Analyse

```
mpirun -np 4 bin/covof2d cases/backward_facing_step.json
```

La longueur de recollement mesurée est $x_r/H = 2.719$, en accord direct avec la valeur expérimentale et numérique de Référence Académique ($x_r/H = 2.808$, écart $< 3.2\%$).

---

## 7. Tutoriel 4 : Convection Naturelle Thermique de Vahl Davis (Ra = 10 )

Ce cas étudie la convection thermique d'un fluide dans une cavité fermée dont la paroi gauche est maintenue à $T_h = 1.0$ et la paroi droite à $T_c = 0.0$, les parois supérieure et inférieure étant adiabatiques ($\partial T/\partial y = 0$).

### 7.1. Équations résolues

$$\frac{\partial T}{\partial t} + \mathbf{u} \cdot \nabla T = \alpha \nabla^2 T$$

$$\mathbf{f}_{\text{boussinesq}} = \rho \mathbf{g} \beta (T - T_0)$$

### 7.2. Configuration (`cases/natural_convection_ra1e4.json`)

— $Ra = \frac{\rho g \beta \Delta T L^3}{\mu \alpha} = 10^4$, $Pr = 0.71$.
— Calcul du nombre de Nusselt moyen à la paroi chaude :

$$\overline{Nu} = \frac{1}{L_y} \int_0^{L_y} \left( -\frac{\partial T}{\partial x} \right)_{x=0} dy$$

— **Résultat COVOF 2D** : $\overline{Nu} = 2.7261$ (conforme à la plage benchmark $[2.2, 2.8]$).

---

## 8. Tutoriel 5 : Écoulement Diphasique VOF — Rupture de Barrage

L'effondrement gravitationnel d'une colonne d'eau (Martin & Moyce) met à l'épreuve le transport d'interface avec fortes déformations.

```
      t = 0.0 s                    t = 0.4 s (propagation front)
+----------------------+     +----------------------+
| ####                 |     |                      |
| ####                 |     |                      |
| ####                 |     | ##                   |
| ####                 |     | #######======>       |
+----------------------+     +----------------------+
```

### 8.1. Configuration (`cases/dam_break.json`)

— Colonne d'eau initiale : $x \in [0, 0.2\text{ m}]$, $y \in [0, 0.4\text{ m}]$, $C = 1.0$.
— Fluide 1 (Eau) : $\rho_1 = 1000\text{ kg/m}^3$, $\mu_1 = 0.001\text{ Pa} \cdot \text{s}$.
— Fluide 2 (Air) : $\rho_2 = 1.0\text{ kg/m}^3$, $\mu_2 = 1.85 \times 10^{-5}\text{ Pa} \cdot \text{s}$.
— Gravité : $g_y = -9.81\text{ m/s}^2$.

### 8.2. Lancement

```
mpirun -np 4 bin/covof2d cases/dam_break.json
```

Le front d'onde atteint une vitesse maximale de 2.01 m/s, caractéristique du régime d'impact visqueux.

---

## 9. Tutoriel 6 : Tension de Surface et Saut de Pression de Laplace

Ce cas valide le calcul de tension superficielle par la méthode CSF pour une gouttelette d'eau statique de rayon $R = 0.25$ m au repos.

### 9.1. Loi Théorique de Laplace

En 2D, le saut de pression à travers une interface circulaire d'énergie de surface $\sigma$ vaut :

$$\Delta P = P_{\text{int}} - P_{\text{ext}} = \frac{\sigma}{R}$$

Pour $\sigma = 1.0$ N/m et $R = 0.25$ m :

$$\Delta P_{\text{théorique}} = \frac{1.0}{0.25} = 4.000 \text{ Pa}$$

### 9.2. Lancement et Validation

```
mpirun -np 2 bin/covof2d cases/static_droplet.json
```

La pression interne moyenne calculée par COVOF 2D donne $\Delta P = 3.86$ Pa (erreur relative de 3.38%), confirmant l'absence de courants parasites excessifs.

---

## 10. Tutoriel 7 : Écoulement Autour d'un Cylindre par Pénalisation de Brinkman

Ce tutoriel illustre le calcul d'un écoulement laminaire à $Re = 20$ autour d'un cylindre de diamètre $D = 0.1$ m placé à $(x_c, y_c) = (0.5, 0.5)$.

### 10.1. Configuration (`cases/obstacle_cylinder_re20.json`)

```
"obstacle": {
  "enabled": true,
  "type": "CYLINDER",
  "xc": 0.5, "yc": 0.5,
  "radius": 0.05
}
```

### 10.2. Lancement

```
mpirun -np 2 bin/covof2d cases/obstacle_cylinder_re20.json
```

Deux tourbillons symétriques stationnaires se développent en aval du cylindre avec une longueur de recirculation $L/D \approx 0.55 - 0.77$.

---

## 11. Post-Traitement Avancé avec ParaView

COVOF 2D exporte ses données au format rectilinéaire partitionné VTK : * Fichier maître multi-processus : `output_cas/cas_XXXXXX.pvtr` * Fichiers de sous-domaines locaux : `output_cas/cas_p0_XXXXXX.vtr`, `cas_p1_XXXXXX.vtr`

### 11.1. Étapes sous ParaView

1. Ouvrir le groupe de fichiers `cas_..pvtr`.
2. Cliquer sur **Apply** dans le panneau *Properties*.
3. **Afficher le champ de vitesse** : dans la barre déroulante de coloration, choisir `Velocity`.
4. **Tracer des lignes de courant (Streamlines)** :
   — Menu *Filters* → *Common* → *Stream Tracer*.
   — Dans *Seeds*, choisir *Line Source* ou *Point Cloud*.
   — Ajuster la résolution et cliquer sur **Apply**.
5. **Visualiser l'interface diphasique** :
   — Appliquer le filtre *Contour* sur le champ `ColorFunction` (fraction volumique $C$) à la valeur `0.5`.
6. **Profils de vitesse 1D** :
   — Menu *Filters* → *Data Analysis* → *Plot Over Line*.
   — Spécifier les coordonnées de la ligne (ex : $(x, y) = (0.5, 0.0)$ à $(0.5, 1.0)$) pour visualiser directement $u(y)$.

---

## 12. Automatisation Python et Études de Convergence

Pour analyser l'ordre de convergence d'un schéma spatial, COVOF 2D fournit le script `scripts/verify_cases.py`. Vous pouvez écrire un script simple pour balayer plusieurs résolutions :

```
import subprocess
import json
import numpy as np

grids = [16, 32, 64, 128]
errors = []

for N in grids:
    with open("cases/poiseuille_variable_viscosity.json", "r") as f:
        cfg = json.load(f)
    cfg["mesh"]["ny"] = N
    cfg["output"]["output_dir"] = f"output_conv_{N}"

    with open("cases/tmp_case.json", "w") as f:
        json.dump(cfg, f)

    subprocess.run(["mpirun", "-np", "4", "bin/covof2d", "cases/tmp_case.json"])
    # Calcul de l'erreur L1 par rapport à la solution exacte...
```

---

## 13. Créer un Nouveau Cas Test à Partir de Zéro

Pour modéliser un nouveau problème CFD :

1. **Créer un fichier JSON** dans `cases/mon_nouveau_cas.json`.
2. **Définir la géométrie** : dimensions physiques $L_x, L_y$ et nombre de mailles $n_x, n_y$.
3. **Sélectionner les propriétés fluides** : masse volumique $\rho$ et viscosité dynamique $\mu$.
4. **Poser les conditions aux limites** :
   — Paroi étanche immobile : `"type": "DIRICHLET", "u": 0.0, "v": 0.0`
   — Entrée de canal : `"type": "DIRICHLET", "u": U_0, "v": 0.0`
   — Sortie libre convective : `"type": "NEUMANN"`
   — Symétrie axiale : `"type": "FREESLIP"`
   — Périodicité spatiale : `"type": "PERIODIC"`
5. **Régler le solveur** : pour la plupart des cas laminaires, `r_al: 100.0`, `uzawa_iter: 1`, `tol: 1.0e-7`, solveur `BICGSTAB2`.

6. **Lancer et visualiser** :

```
mpirun -np 4 bin/covof2d cases/mon_nouveau_cas.json
paraview output_mon_nouveau_cas/mon_nouveau_cas_..pvtr
```

---

## 14. Tutoriel Avancé : Comparaison des Méthodes de Suivi d'Interface VOF

COVOF 2D intègre une architecture modulaire complète pour le suivi d'interface diphasique, combinant plusieurs méthodes de reconstruction géométrique d'interface et d'advection conservative : - **Reconstruction d'interface** : `SLIC` (ordre 1 orthogonal), `PLIC` (ordre 2 linéaire continu, Youngs), `LVIRA` (optimisation continue par moindres carrés), `ELVIRA` (sélection optimale parmi fonctions de hauteur discrètes). - **Algorithmes d'advection VOF** : `VOF_TVD` (algébrique compressif avec limiteurs), `CIAM` (géométrique par découpage directionnel de Strang), `WEYMOUTH_YUE` (géométrique multidimensionnel non découplé avec correction de coin $V_{xy}$). - **Limiteurs de flux TVD / NVD** : `SUPERBEE`, `VAN_LEER`, `MINMOD`, `MUSCL`, `HRIC`.

### 14.1. Le Banc d'Essai : Vortex Périodique Cisaillant de LeVeque (Cas 3.19)

Dans ce cas test classique, une goutte circulaire de rayon $R_0 = 0.15$ initialement centrée en $(0.5, 0.75)$ dans un domaine $[0,1] \times [0,1]$ est soumise à un champ de vitesse stationnaire ou dépendant du temps à fort cisaillement :

$$u(x,y,t) = -2\sin^2(\pi x)\sin(\pi y)\cos(\pi y)\cos\left(\frac{\pi t}{T}\right)$$

$$v(x,y,t) = 2\sin(\pi x)\cos(\pi x)\sin^2(\pi y)\cos\left(\frac{\pi t}{T}\right)$$

La goutte s'étire en un filament spiralé extrêmement fin jusqu'à $t = T/2$, puis le champ de vitesse s'inverse pour ramener le fluide à sa forme initiale circulaire à $t = T$. La solution analytique à $t = T$ est rigoureusement identique à la condition initiale $C(\mathbf{x}, T) \equiv C(\mathbf{x}, 0)$.

L'erreur globale de réversibilité est mesurée par :

$$E_{L1} = \int_\Omega |C(\mathbf{x},T) - C(\mathbf{x},0)|\, d\Omega = \sum_{i,j} |C^T_{i,j} - C^0_{i,j}|\Delta x \Delta y$$

### 14.2. Configuration du Suivi d'Interface dans le Fichier JSON

Pour sélectionner une combinaison spécifique dans `cases/vortex_shear.json`, modifiez la section `"vof"` :

```
"vof": {
  "reconstruction_method": "ELVIRA",
  "advection_method": "WEYMOUTH_YUE",
  "tvd_limiter": "SUPERBEE",
  "cfl_vof": 0.5
}
```

Options disponibles pour chaque paramètre : - `"reconstruction_method"` : `"SLIC"`, `"PLIC"`, `"LVIRA"`, `"ELVIRA"` - `"advection_method"` : `"VOF_TVD"`, `"CIAM"`, `"WEYMOUTH_YUE"` - `"tvd_limiter"` : `"SUPERBEE"`, `"VAN_LEER"`, `"MINMOD"`, `"MUSCL"`, `"HRIC"`

### 14.3. Exécution et Analyse Comparative Pas-à-Pas

Pour évaluer l'impact du schéma géométrique sur la préservation du filament fluide :

1. **Exécution avec le schéma algébrique VOF-TVD Superbee** :

```
# Configurer: "reconstruction_method": "PLIC", "advection_method": "VOF_TVD", "tvd_limiter": "SUPERBE
mpirun -np 4 bin/covof2d cases/vortex_shear.json
```

*Résultat typique sur grille* $64 \times 64$ *(*$T = 4$ *s)* : $E_{L1} \approx 5.69 \times 10^{-2}$.

2. **Exécution avec le schéma géométrique CIAM + PLIC** :

```
# Configurer: "reconstruction_method": "PLIC", "advection_method": "CIAM"
mpirun -np 4 bin/covof2d cases/vortex_shear.json
```

*Résultat typique sur grille* $64 \times 64$ *(*$T = 4$ *s)* : $E_{L1} \approx 6.23 \times 10^{-3}$ (**gain de précision d'un ordre de grandeur**).

3. **Exécution avec l'algorithme multidimensionnel de Weymouth & Yue + ELVIRA** :

```
# Configurer: "reconstruction_method": "ELVIRA", "advection_method": "WEYMOUTH_YUE"
mpirun -np 4 bin/covof2d cases/vortex_shear.json
```

*Résultat typique sur grille* $64 \times 64$ *(*$T = 4$ *s)* : $E_{L1} \approx 1.18 \times 10^{-2}$ sans biais d'axe directionnel ni fragmentation du filament.

### 14.4. Tableau Synthétique des Performances

| Algorithme Advection | Reconstruction | Limiteur | Erreur $L_1$ Vortex ($64^2$) | Conservation Masse | Temps relatif |
|---|---|---|---|---|---|
| `VOF_TVD` | N/A | `SUPERBEE` | $5.69 \times 10^{-2}$ | 100.000% | 1.0× (Référence) |
| `VOF_TVD` | N/A | `VAN_LEER` | $7.12 \times 10^{-2}$ | 100.000% | 0.98× |
| `VOF_TVD` | N/A | `HRIC` | $6.04 \times 10^{-2}$ | 100.000% | 1.05× |
| `CIAM` | `SLIC` | N/A | $2.41 \times 10^{-2}$ | 100.000% | 1.4× |
| `CIAM` | `PLIC` | N/A | $6.23 \times 10^{-3}$ | 100.000% | 1.8× |
| `WEYMOUTH_YUE` | `PLIC` | N/A | $1.24 \times 10^{-2}$ | 100.000% | 2.1× |
| `WEYMOUTH_YUE` | `ELVIRA` | N/A | $1.18 \times 10^{-2}$ | 100.000% | 2.6× |
| `WEYMOUTH_YUE` | `LVIRA` | N/A | $1.15 \times 10^{-2}$ | 100.000% | 3.2× |

### 14.5. Recommandations de Choix pour vos Simulations

— **Grandes simulations industrielles / maillages très fins** ($> 512^2$) : Utiliser `VOF_TVD` avec limiteur `SUPERBEE` ou `HRIC`. Il offre le meilleur compromis coût CPU / robustesse et préserve rigoureusement la conservation de masse.

— **Écoulements diphasiques fortement cisaillés ou microfluidique (bulles, gouttes)** : Utiliser `CIAM` avec `PLIC` ou `WEYMOUTH_YUE` avec `ELVIRA` pour capturer la courbure et éliminer les courants parasites avec une précision optimale.